<?php

MATERIAL DE ESTUDIO / DESARROLLO WEB

UNIDAD DIDÁCTICA 02

Tutorial 07
Catálogo de películas

Usuarios, comentarios, valoraciones y organización por capas

PHP · PDO · Sesiones · Capas

APUNTES DE CLASE

Desarrollo web · PHP · Tutorial guiado

{ PHP }

UD 02 · DWES · PHP 8.1+

Tutorial 07 · Aplicación de películas por capas

Objetivo: Vamos a integrar los conceptos anteriores en un catálogo de películas con usuarios, perfiles de administración, comentarios y valoraciones. Construiremos primero la base de datos, después el repositorio y por último la página que recibe las peticiones y muestra las vistas. La aplicación utiliza una separación de responsabilidades **simplificada**: el modelo está en `model/Repositorio.php` y `index.php` reúne control y presentación.

Antes de comenzar

Utiliza PHP 8.1 o posterior y un servidor web local (por ejemplo, Apache de XAMPP). Crea una carpeta de proyecto dentro de `htdocs/UD2/` y **escribe los archivos con el contenido mostrado en cada paso**. El directorio `Codigo/` contiene esos mismos archivos listos para ejecutar.

Inicia **Apache y MySQL** desde XAMPP. Comprueba que PHP dispone de la extensión `pdo_mysql`. En los ejemplos se usa MySQL local con usuario `root` y contraseña vacía, una configuración habitual de laboratorio: si tu instalación utiliza otras credenciales, cámbialas en `config.php`. No utilices esas credenciales de prueba en un servidor público.

Crea `htdocs/UD2/Peliculas/`. La estructura será:

```
Peliculas/
├── schema.sql
├── config.php
├── model/
│   └── Repositorio.php
├── crear_admin.php
└── index.php
```

Accederás a `http://localhost/UD2/Peliculas/index.php`. Para crear la cuenta de administrador necesitarás abrir una terminal en la carpeta de proyecto y disponer del comando `php` (en XAMPP Windows puede encontrarse en `C:\xampp\php\php.exe`).

Paso 1. Crear las cuatro tablas de la base de datos

Abre phpMyAdmin, entra en la pestaña SQL y ejecuta `schema.sql`. Se crea la base de datos `ud2_peliculas` y cuatro tablas:

- `usuario` guarda correo, nombre, hash de contraseña y rol (`ADMIN` o `USUARIO`).
- `pelicula` contiene título, sinopsis y año.
- `comentario` guarda un texto relacionado con una película y su autor.
- `valoracion` guarda una nota de un usuario para una película.

Las claves foráneas enlazan los comentarios y valoraciones con sus autores y películas. La clave primaria doble de `valoracion (pelicula_id, usuario_id)` impide que una misma persona tenga dos valoraciones distintas sobre la misma película. Con `ON DELETE CASCADE`, cuando se borra una película se borran también sus comentarios y valoraciones.

Archivo `Codigo/schema.sql`

```
CREATE DATABASE IF NOT EXISTS ud2_peliculas
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE ud2_peliculas;

CREATE TABLE IF NOT EXISTS usuario (
    id INT AUTO_INCREMENT PRIMARY KEY,
    email VARCHAR(160) NOT NULL UNIQUE,
    nombre VARCHAR(100) NOT NULL,
    hash VARCHAR(255) NOT NULL,
    rol ENUM('ADMIN', 'USUARIO') NOT NULL DEFAULT 'USUARIO'
);

CREATE TABLE IF NOT EXISTS pelicula (
    id INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    sinopsis TEXT NOT NULL,
    anio SMALLINT NOT NULL
);

CREATE TABLE IF NOT EXISTS comentario (
    id INT AUTO_INCREMENT PRIMARY KEY,
    pelicula_id INT NOT NULL,
    usuario_id INT NOT NULL,
    texto TEXT NOT NULL,
    FOREIGN KEY (pelicula_id) REFERENCES pelicula(id) ON DELETE CASCADE,
    FOREIGN KEY (usuario_id) REFERENCES usuario(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS valoracion (
    pelicula_id INT NOT NULL,
    usuario_id INT NOT NULL,
    nota TINYINT NOT NULL,
    PRIMARY KEY (pelicula_id, usuario_id),
    FOREIGN KEY (pelicula_id) REFERENCES pelicula(id) ON DELETE CASCADE,
    FOREIGN KEY (usuario_id) REFERENCES usuario(id) ON DELETE CASCADE,
    CONSTRAINT chk_nota CHECK (nota BETWEEN 1 AND 5)
);
```

La tabla `valoracion` admite notas de 1 a 5. Su comprobación `CHECK` es una protección adicional a la validación que haremos en PHP.

Paso 2. Preparar la conexión en `config.php`

Crea `config.php` en la raíz del proyecto. `conectar()` devuelve una conexión PDO con codificación `utf8mb4`, excepciones para errores SQL y resultados asociativos. La función `h()` transforma texto en salida HTML segura mediante `htmlspecialchars()`.

`h()` se usará al mostrar nombres, títulos, comentarios y sinopsis. No se usa en el repositorio porque su trabajo es gestionar datos, no producir HTML.

Archivo `Codigo/config.php`

```
<?php
declare(strict_types=1);

function conectar(): PDO
{
    return new PDO(
```

```
        'mysql:host=127.0.0.1;dbname=ud2_peliculas;charset=utf8mb4',
        'root',
        '',
        [
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            PDO::ATTR_EMULATE_PREPARES => false
        ]
    );
}

function h(string $valor): string
{
    return htmlspecialchars($valor, ENT_QUOTES, 'UTF-8');
}
```

Si falla la conexión, revisa el nombre `ud2_peliculas`, que MySQL está iniciado y las credenciales del equipo.

Paso 3. Crear el modelo `Repositorio.php`

Crea la subcarpeta `model` y dentro `Repositorio.php`. El constructor recibe una conexión PDO. Aquí están juntas todas las operaciones con la base de datos:

- `buscarUsuario()` busca un correo y devuelve su registro o `false`.
- `registrar()` valida y crea un usuario con rol `USUARIO` y contraseña convertida a hash.
- `peliculas()` lista las películas junto a la media de sus valoraciones mediante `LEFT JOIN` y `AVG`.
- `pelicula()` y `comentarios()` recuperan el detalle y sus comentarios.
- `guardarPelicula()` realiza `INSERT` si el ID es `null` y `UPDATE` si hay ID.
- `borrarPelicula()` elimina por ID.
- `comentar()` añade un comentario asociado a usuario y película.
- `valorar()` inserta una nota o actualiza la que ese usuario ya tenía.

Todas las operaciones que incorporan datos externos utilizan consultas preparadas. El modelo no comprueba la sesión de usuario ni imprime HTML; esas tareas estarán en el archivo de entrada.

Archivo `Codigo/model/Repositorio.php`

```
<?php
declare(strict_types=1);

class Repositorio
{
    public function __construct(private PDO $db) {}

    public function buscarUsuario(string $email): array|false
    {
        $sentencia = $this->db->prepare(
            'SELECT id, nombre, hash, rol FROM usuario WHERE email = ?'
        );
        $sentencia->execute([strtolower(trim($email))]);
        return $sentencia->fetch();
    }

    public function registrar(string $nombre, string $email, string $clave): void
    {
        $nombre = trim($nombre);
        $email = strtolower(trim($email));
        if ($nombre === '' || strlen($nombre) > 100
            || !filter_var($email, FILTER_VALIDATE_EMAIL) || strlen($clave) < 8) {
```

```
            throw new InvalidArgumentException('Datos de registro no válidos.');
        }
        $sentencia = $this->db->prepare(
            'INSERT INTO usuario(nombre, email, hash, rol) VALUES (?, ?, ?, ?)'
        );
        $sentencia->execute([
            $nombre, $email, password_hash($clave, PASSWORD_DEFAULT), 'USUARIO'
        ]);
    }

    public function peliculas(): array
    {
        $sql = 'SELECT p.*, ROUND(AVG(v.nota), 1) AS media '
            . 'FROM pelicula p '
            . 'LEFT JOIN valoracion v ON v.pelicula_id = p.id '
            . 'GROUP BY p.id ORDER BY p.id DESC';
        return $this->db->query($sql)->fetchAll();
    }

    public function pelicula(int $id): array|false
    {
        $sentencia = $this->db->prepare(
            'SELECT id, titulo, sinopsis, anio FROM pelicula WHERE id = ?'
        );
        $sentencia->execute([$id]);
        return $sentencia->fetch();
    }

    public function comentarios(int $id): array
    {
        $sql = 'SELECT c.texto, u.nombre '
            . 'FROM comentario c JOIN usuario u ON u.id = c.usuario_id '
            . 'WHERE c.pelicula_id = ? ORDER BY c.id DESC';
        $sentencia = $this->db->prepare($sql);
        $sentencia->execute([$id]);
        return $sentencia->fetchAll();
    }

    public function guardarPelicula(?int $id, string $titulo, string $sinopsis, int $anio): void
    {
        $titulo = trim($titulo);
        $sinopsis = trim($sinopsis);
        if ($titulo === '' || strlen($titulo) > 150 || $sinopsis === ''
            || $anio < 1895 || $anio > 2100) {
            throw new InvalidArgumentException('Datos de película no válidos.');
        }
        if ($id === null) {
            $sentencia = $this->db->prepare(
                'INSERT INTO pelicula(titulo, sinopsis, anio) VALUES (?, ?, ?)'
            );
            $sentencia->execute([$titulo, $sinopsis, $anio]);
        } else {
            $sentencia = $this->db->prepare(
                'UPDATE pelicula SET titulo = ?, sinopsis = ?, anio = ? WHERE id = ?'
            );
            $sentencia->execute([$titulo, $sinopsis, $anio, $id]);
        }
    }

    public function borrarPelicula(int $id): void
    {
        $sentencia = $this->db->prepare('DELETE FROM pelicula WHERE id = ?');
        $sentencia->execute([$id]);
    }

    public function comentar(int $id, int $usuario, string $texto): void
    {
        $texto = trim($texto);
```

```
        if ($texto === '' || strlen($texto) > 1000) {
            throw new InvalidArgumentException('Comentario no válido.');
        }
        $sentencia = $this->db->prepare(
            'INSERT INTO comentario(pelicula_id, usuario_id, texto) VALUES (?, ?, ?)'
        );
        $sentencia->execute([$id, $usuario, $texto]);
    }

    public function valorar(int $id, int $usuario, int $nota): void
    {
        if ($nota < 1 || $nota > 5) {
            throw new InvalidArgumentException('La nota debe estar entre 1 y 5.');
        }
        $sql = 'INSERT INTO valoracion(pelicula_id, usuario_id, nota) VALUES (?, ?, ?) '
            . 'ON DUPLICATE KEY UPDATE nota = VALUES(nota)';
        $sentencia = $this->db->prepare($sql);
        $sentencia->execute([$id, $usuario, $nota]);
    }
}
```

En `peliculas()`, el `LEFT JOIN` permite que también aparezcan películas que todavía no tienen ninguna valoración. En ese caso `AVG` devuelve `NULL` y la página mostrará «sin valorar». La consulta de `valorar()` aprovecha la clave primaria compuesta: `ON DUPLICATE KEY UPDATE` sustituye la nota anterior por la nueva.

Paso 4. Preparar una cuenta administradora local

Crea `crear_admin.php`. Este archivo se ejecuta **una vez desde la terminal**, después de importar el SQL. Comprueba `PHP_SAPI !== 'cli'` para impedir que se use desde el navegador. Se crea o actualiza un administrador de prueba con correo `admin@cine.test` y contraseña `Admin2026!`, siempre guardada mediante `password_hash()`.

Sitúate en la carpeta `Peliculas/` y ejecuta:

```
php crear_admin.php
```

Si trabajas con XAMPP en Windows y no tienes PHP en el PATH, utiliza la ruta completa del ejecutable de PHP. La contraseña es exclusivamente para el equipo local de clase; no se debe conservar en aplicaciones públicas.

Archivo `Codigo/crear_admin.php`

```
<?php
declare(strict_types=1);

// Ejecutar solo desde terminal en el laboratorio: php crear_admin.php
if (PHP_SAPI !== 'cli') {
    http_response_code(403);
    exit;
}
require_once __DIR__ . '/config.php';

$email = 'admin@cine.test';
$clave = 'Admin2026!';
$sql = 'INSERT INTO usuario(nombre, email, hash, rol) VALUES (?, ?, ?, ?) '
    . 'ON DUPLICATE KEY UPDATE hash = VALUES(hash), rol = VALUES(rol)';
$sentencia = conectar()->prepare($sql);
$sentencia->execute([
    'Administrador', $email, password_hash($clave, PASSWORD_DEFAULT), 'ADMIN'
]);
echo "Cuenta de prueba: $email / $clave" . PHP_EOL;
```

Tras ejecutar el comando, phpMyAdmin mostrará una fila en `usuario` con `rol=ADMIN`; la columna `hash` contendrá un hash, no la contraseña original.

Paso 5. Crear `index.php`: solicitudes, permisos y vistas

Este archivo es la entrada de todas las peticiones web. Vamos a escribirlo completo tal como aparece debajo. Para comprenderlo, sigue el orden de ejecución:

1. Inicio y datos comunes. `session_start()` permite utilizar `$_SESSION`. Se cargan `config.php` y el modelo. Se preparan las variables `$error`, `$peliculas`, `$pelicula` y `$comentarios`.

2. Procesamiento de formularios. Si el método es `POST`, el campo oculto `accion` indica qué operación se solicita. En `login`, se busca el usuario, se verifica su hash y se guarda en sesión únicamente ID, nombre y rol. En `registro`, se llama al repositorio; el usuario nuevo obtiene el rol ordinario y deberá iniciar sesión después. En `logout`, se vacía la sesión de la página.

3. Control de permisos. Para guardar o eliminar películas se exige `rol=ADMIN` en el servidor. Para comentar y valorar se exige que exista un usuario autenticado. No basta con ocultar botones: cualquier persona puede enviar un POST manual, por eso los permisos se verifican también en la rama PHP.

4. Actualización y redirección. Las operaciones de escritura terminan con `header('Location: ...')` y `exit`. Para comentarios y valoraciones se vuelve al detalle de la misma película; para las otras operaciones, al catálogo.

5. Consulta y presentación. En `GET`, `peliculas()` recupera el catálogo. Si llega `?id=5`, también se consultan esa película y sus comentarios. El HTML muestra formularios de acceso y registro, el formulario de administración únicamente para `ADMIN`, el catálogo para todos y los formularios de comentario y valoración para usuarios autenticados. Los valores de texto pasan por `h()` antes de mostrarse.

A continuación aparece `index.php`, con la lógica de peticiones y el HTML de la página.

Archivo `Codigo/index.php`

```
<?php
declare(strict_types=1);
session_start();
require_once __DIR__ . '/config.php';
require_once __DIR__ . '/model/Repositorio.php';

$error = '';
$peliculas = [];
$pelicula = false;
$comentarios = [];

try {
    $repo = new Repositorio(conectar());

    if ($_SERVER['REQUEST_METHOD'] === 'POST') {
        $accion = (string) ($_POST['accion'] ?? '');
        $id = filter_var(
            $_POST['id'] ?? null,
            FILTER_VALIDATE_INT,
            ['options' => ['min_range' => 1]]
        );
        $usuario = $_SESSION['usuario'] ?? null;

        if ($accion === 'login') {
            $encontrado = $repo->buscarUsuario((string) ($_POST['email'] ?? ''));
            $clave = (string) ($_POST['clave'] ?? '');
            if (!$encontrado || !password_verify($clave, $encontrado['hash'])) {
                throw new InvalidArgumentException('Acceso incorrecto.');
            }
```

```
            session_regenerate_id(true);
            $_SESSION['usuario'] = [
                'id' => (int) $encontrado['id'],
                'nombre' => $encontrado['nombre'],
                'rol' => $encontrado['rol']
            ];
        } elseif ($accion === 'registro') {
            $repo->registrar(
                (string) ($_POST['nombre'] ?? ''),
                (string) ($_POST['email'] ?? ''),
                (string) ($_POST['clave'] ?? '')
            );
        } elseif ($accion === 'logout') {
            $_SESSION = [];
            session_regenerate_id(true);
        } elseif ($accion === 'guardar' && ($usuario['rol'] ?? '') === 'ADMIN') {
            $anio = filter_var($_POST['anio'] ?? null, FILTER_VALIDATE_INT);
            if ($anio === false) {
                throw new InvalidArgumentException('Año no válido.');
            }
            // ID vacío significa alta; ID numérico válido significa edición.
            $idTexto = trim((string) ($_POST['id'] ?? ''));
            if ($idTexto !== '' && $id === false) {
                throw new InvalidArgumentException('ID de película no válido.');
            }
            $repo->guardarPelicula(
                $idTexto === '' ? null : $id,
                (string) ($_POST['titulo'] ?? ''),
                (string) ($_POST['sinopsis'] ?? ''),
                $anio
            );
        } elseif ($accion === 'eliminar' && $id !== false
            && ($usuario['rol'] ?? '') === 'ADMIN') {
            $repo->borrarPelicula($id);
        } elseif ($accion === 'comentar' && $id !== false && $usuario) {
            $repo->comentar($id, (int) $usuario['id'], (string) ($_POST['texto'] ?? ''));
        } elseif ($accion === 'valorar' && $id !== false && $usuario) {
            $nota = filter_var($_POST['nota'] ?? null, FILTER_VALIDATE_INT);
            if ($nota === false) {
                throw new InvalidArgumentException('Nota no válida.');
            }
            $repo->valorar($id, (int) $usuario['id'], $nota);
        } else {
            throw new InvalidArgumentException('Acción no permitida.');
        }

        $volver = ($id !== false && in_array($accion, ['comentar', 'valorar'], true))
            ? 'index.php?id=' . $id : 'index.php';
        header('Location: ' . $volver);
        exit;
    }

    $peliculas = $repo->peliculas();
    $detalle = filter_var(
        $_GET['id'] ?? null,
        FILTER_VALIDATE_INT,
        ['options' => ['min_range' => 1]]
    );
    if ($detalle !== false) {
        $pelicula = $repo->pelicula($detalle);
        if ($pelicula) {
            $comentarios = $repo->comentarios($detalle);
        }
    }
} catch (InvalidArgumentException $errorValidacion) {
    http_response_code(400);
    $error = $errorValidacion->getMessage();
} catch (PDOException $errorBD) {
```

```
    http_response_code(500);
    $error = 'Error en la operación o la base de datos.';
}
?>
<!doctype html>
<html lang="es">
<head>
    <meta charset="utf-8">
    <title>Películas</title>
</head>
<body>
    <h1>Películas</h1>
    <p><a href="index.php">Inicio</a></p>
    <?php if ($error !== ''): ?>
        <p><?= h($error) ?></p>
    <?php endif; ?>

    <?php if (!isset($_SESSION['usuario'])): ?>
        <h2>Iniciar sesión</h2>
        <form action="index.php" method="post">
            <input type="hidden" name="accion" value="login">
            <input type="email" name="email" placeholder="Correo" required>
            <input type="password" name="clave" placeholder="Contraseña" required>
            <button type="submit">Entrar</button>
        </form>
        <h2>Registrarse</h2>
        <form action="index.php" method="post">
            <input type="hidden" name="accion" value="registro">
            <input name="nombre" maxlength="100" placeholder="Nombre" required>
            <input type="email" name="email" placeholder="Correo" required>
            <input type="password" name="clave" minlength="8" placeholder="Clave" required>
            <button type="submit">Registrarse</button>
        </form>
    <?php else: ?>
        <p>
            <?= h($_SESSION['usuario']['nombre']) ?>
            (<?= h($_SESSION['usuario']['rol']) ?>)
        </p>
        <form action="index.php" method="post">
            <input type="hidden" name="accion" value="logout">
            <button type="submit">Cerrar sesión</button>
        </form>
    <?php endif; ?>

    <?php if (($_SESSION['usuario']['rol'] ?? '') === 'ADMIN'): ?>
        <h2>Alta / edición de película</h2>
        <form action="index.php" method="post">
            <input type="hidden" name="accion" value="guardar">
            <label>ID (vacío para alta)
                <input name="id" type="number" min="1">
            </label>
            <label>Título
                <input name="titulo" maxlength="150" required>
            </label>
            <label>Sinopsis
                <textarea name="sinopsis" required></textarea>
            </label>
            <label>Año
                <input name="anio" type="number" min="1895" max="2100" required>
            </label>
            <button type="submit">Guardar</button>
        </form>
    <?php endif; ?>

    <h2>Catálogo</h2>
    <ul>
        <?php foreach ($peliculas as $film): ?>
            <li>
```

```
                <a href="index.php?id=<?= (int) $film['id'] ?>">
                    <?= h($film['titulo']) ?>
                </a>
                (<?= (int) $film['anio'] ?>) —
                media <?= $film['media'] === null ? 'sin valorar' : h((string) $film['media']) ?>
                <?php if (($_SESSION['usuario']['rol'] ?? '') === 'ADMIN'): ?>
                    <form action="index.php" method="post" style="display:inline">
                        <input type="hidden" name="accion" value="eliminar">
                        <input type="hidden" name="id" value="<?= (int) $film['id'] ?>">
                        <button type="submit">Eliminar</button>
                    </form>
                <?php endif; ?>
            </li>
        <?php endforeach; ?>
    </ul>

    <?php if ($pelicula): ?>
        <h2><?= h($pelicula['titulo']) ?></h2>
        <p><?= nl2br(h($pelicula['sinopsis'])) ?></p>
        <?php if (isset($_SESSION['usuario'])): ?>
            <h3>Nuevo comentario</h3>
            <form action="index.php" method="post">
                <input type="hidden" name="accion" value="comentar">
                <input type="hidden" name="id" value="<?= (int) $pelicula['id'] ?>">
                <textarea name="texto" maxlength="1000" required></textarea>
                <button type="submit">Comentar</button>
            </form>
            <h3>Tu valoración</h3>
            <form action="index.php" method="post">
                <input type="hidden" name="accion" value="valorar">
                <input type="hidden" name="id" value="<?= (int) $pelicula['id'] ?>">
                <input type="number" name="nota" min="1" max="5" required>
                <button type="submit">Valorar</button>
            </form>
        <?php endif; ?>
        <h3>Comentarios</h3>
        <ul>
            <?php foreach ($comentarios as $comentario): ?>
                <li><?= h($comentario['nombre']) ?>: <?= h($comentario['texto']) ?></li>
            <?php endforeach; ?>
        </ul>
    <?php endif; ?>
</body>
</html>
```

El formulario de administración utiliza un campo ID: vacío crea una película nueva; con el ID de una película existente la actualiza. El usuario ordinario no podrá ejecutar esas acciones ni aunque manipule el campo oculto `accion`.

Resultado y comprobación

Sigue esta secuencia para ver las funcionalidades ya programadas:

1. Entra en `index.php` sin iniciar sesión: aparecen acceso, registro y catálogo vacío, pero no el formulario de administración.
2. Inicia sesión como `admin@cine.test` / `Admin2026!`. Aparece el formulario «Alta / edición de película». Deja el ID vacío y registra `La película de prueba`, sinopsis `Ejemplo de catálogo` y año `2024`.
3. Abre el enlace de la película y observa su ID en la dirección (`?id=...`). Vuelve al catálogo y usa ese ID en el formulario de administración para cambiar el título o la sinopsis. El formulario actualiza el registro en lugar de crear otro.

4. Cierra sesión, registra `ana@cine.test` con una contraseña de al menos ocho caracteres e inicia sesión con esa cuenta. Puedes abrir el detalle, comentar y puntuar de 1 a 5; no aparece el formulario de administración.
5. Valora dos veces la misma película con notas distintas: la nota se **actualiza**, no se crea una segunda fila en `valoracion`. La media del catálogo se recalcula.
6. Vuelve a entrar como administrador y elimina la película. Sus comentarios y valoraciones desaparecen por las claves foráneas con `ON DELETE CASCADE`.

Lo importante: el repositorio concentra el SQL; la página recibe las peticiones, comprueba los permisos y genera las vistas. El ejemplo es local y didáctico, no un MVC separado en tres capas independientes ni una aplicación preparada para Internet. En un despliegue real habría que reforzar el cierre de sesión, añadir protección CSRF, limitar intentos de acceso y configurar HTTPS.